

Birzeit University

Faculty of Engineering & Technology

Computer Science Department

Software Engineering - COMP433

Group G1

FINAL (COMPLETE PROJECT) REPORT

Project Title: *Online Water Shop*

Prepared by:

Ahmad Elayyan (Project Manager) - 1210443

Abdallah AboShoaib (Technical Architect) - 1210211

Mohammad Obiad (Secretary/Developer) - 1210093

Mohmmad ALabed (Programmer/Tester) - 1201397

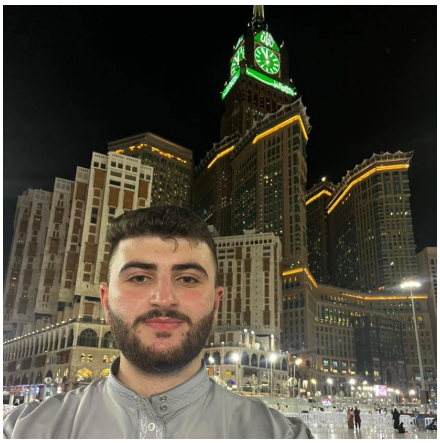
Section: 1

Instructor: Prof. Adel Taweel

Date: Thursday 28th August, 2025

Team Members

Group G1 – Online Water Shop



Ahmad Elayyan
1210443



Abdallah AboShoaib
1210211



Mohammad Obiad
1210093



Mohmmad ALabed
1201397

Contents

1 Chapter 1: Project Planning and Management **3**
- 1.1 Names of Editors/writers of the Report . . . 3
- 1.2 Business Title . . . 3
- 1.3 Group Name: Group Number . . . 3
- 1.4 Name of students/members . . . 3
- 1.5 Role of each member . . . 3
- 1.6 Project management strategy . . . 3
- 1.7 Project Manager Report . . . 4
- 1.8 Group Members Report . . . 5

2 Chapter 2: Requirement Elicitation, Analysis and Modeling **7**
- 2.1 Requirement Statement / Business Description . . . 7
- 2.2 USER and SYSTEM Requirements . . . 10
- 2.3 SCENARIOs . . . 13
- 2.4 Effort/Time Estimation Calculation [**MODIFIED**] . . . 18
- 2.5 Actor Analysis [**MODIFIED**] . . . 20
- 2.6 USE-CASE Diagram [**MODIFIED**] . . . 21
- 2.7 Description of Key Use-Cases . . . 22
- 2.8 ACTIVITY Diagram [**MODIFIED**] . . . 29
- 2.9 Instance Activity Diagrams . . . 30

3 Chapter 3: System Analysis and Modelling **34**
- 3.1 3.1 System CLASS Diagrams [**MODIFIED**] . . . 34
  - 3.1.1 Analysis Class Diagram . . . 34
  - 3.1.2 Detailed Class Diagram [**MODIFIED**] . . . 36
- 3.2 SEQUENCE Diagrams . . . 37

4 Chapter 4: System Design and Modelling **42**
- 4.1 Description of Chosen Design Goals [**MODIFIED**] . . . 42
- 4.2 Component Diagram of System Components [**MODIFIED**] . . . 45
- 4.3 Overall Architecture Diagram [**MODIFIED**] . . . 47
- 4.4 Deployment Diagram [**MODIFIED**] . . . 49

1 Chapter 1: Project Planning and Management

1.1 Names of Editors/writers of the Report

This report was compiled and written by: **Ahmad Elayyan (1210443)** and **Abdallah AboShoaib (1210211)** and **Mohammed obied (1210093)**.

1.2 Business Title

Online Water Shop

1.3 Group Name: Group Number

Group #G1

1.4 Name of students/members

- Ahmad Elayyan – 1210443
- Abdallah AboShoaib – 1210211
- Mohammad Obiad – 1210093
- Mohmmad ALabed – 1201397

1.5 Role of each member

- Ahmad Elayyan – Project Manager
- Abdallah AboShoaib – Technical Architect
- Mohammad Obiad – Secretary / Developer
- Mohmmad ALabed – Programmer / Tester

1.6 Project management strategy

Meetings: The team planned to meet at least once per week in-person to accomplish key phases and ensure steady progress. However, due to time and location constraints, most of the meetings were conducted online using Discord, supported by shared documents and notes.

Decisions: All decisions were made through group discussions, where each member's opinion was considered. If a disagreement occurred, the team voted; in the case of a tie, the final decision was taken by the Project Manager (Ahmad Elayyan).

Process Model: The team followed a hybrid approach. The **Waterfall** model was applied in early stages such as requirement gathering and planning, while an **Agile-like iterative** method was used in the design and modeling phases to allow frequent updates based on peer and instructor feedback.

1.7 Project Manager Report

This was a major project that required dedication, planning, and full collaboration. Each member handled their individual tasks responsibly and contributed to group-wide tasks with commitment. We used **PlantUML** and **Lucidchart** to collaborate on diagrams in real-time, and shared files through Google Drive and Telegram.

- **Ahmad Elayyan:** As the **Project Manager**, he coordinated meetings, tracked deadlines, and ensured all deliverables were on time. He led the **Use Case Diagram**, contributed to the **Requirement Specification, Sequence Diagram (Bulk Order)**, and reviewed the **Deployment and Component Models**.

- **Abdallah AboShoaib:** As the **Technical Architect**, he led the development of **System Class Diagrams (Analysis and Detailed)**, contributed to the **Activity Diagram**, and reviewed multiple components including **Architecture Design**. Individually, he created the scenario and all models for **"Place Order"**, including **Use Case, Activity, and Sequence Diagram**.

- **Mohammad Obiad:** As the **Secretary and Developer**, he managed the group's document structure, notes, and formatting. He contributed to **Actor Analysis, Feedback Modelling, and System Design Goals**, and wrote the full scenario for **"Refill / Exchange"**, with the **Use Case, Activity, and Sequence Diagram**.

- **Mohmmad ALabed:** As the **Programmer and Tester**, he worked on the modeling of error and refund handling. He contributed to **Component Diagrams, System Requirements, and Testing Strategy**, and wrote the full scenario and models for **"Return / Refund"**, including **Use Case, Activity, and Sequence Diagram**.

this course added valuable experience that mirrors real-world software development. It pushed us to coordinate effectively under time pressure and gave us firsthand understanding of system design, requirement engineering, and modeling standards. We leave this project with stronger skills in **communication, collaboration, and technical design**, all essential for our careers.

1.8 Group Members Report

Technical Architect: Abdallah AboShoaib

This project was a rich and transformative experience. Despite the constraints posed by external circumstances, including time pressure and a difficult political climate, our team managed to maintain steady progress through organized online meetings and a shared sense of responsibility. My role as the **Technical Architect** included leading the **Analysis and Detailed Class Diagrams**, which required ensuring consistency between attributes, operations, and relationships across the system. I reviewed and refined the **Use Case and Activity Diagrams**, and participated in discussions around feedback and actor responsibilities. I led the modeling for the **"Place Order"** scenario, developing its **Use Case Description, Activity Diagram, and Sequence Diagram**. I also contributed significantly to the **Architecture Design**, where I helped ensure that our system followed principles of modularity and cohesion. Overall, this course improved my confidence in handling end-to-end system modeling, and I now better understand how conceptual models become the foundation of real software.

Project Manager: Ahmad Elayyan

Leading this project has been an exceptional learning journey for me. As the **Project Manager**, I was responsible for setting deadlines, managing communications, ensuring accountability, and compiling the final report. I actively participated in the **Use Case Diagram**, and created the full **"Bulk Order"** use case with a focus on **Activity and Sequence diagrams**. I helped coordinate multiple iterations of the Component and Deployment diagrams, while also reviewing every submission to ensure clarity and alignment with the instructor's expectations. Under pressure, I maintained team motivation and aligned our tasks with both the software engineering process and real-world challenges. This experience enhanced my technical and leadership skills, particularly in conflict resolution, planning, and delivering results.

Secretary / Developer: Mohammad Obiad

Serving as both the **Secretary and Developer**, I took on documentation tasks, organizing diagrams, and ensuring clarity in naming and labeling across all deliverables. I also handled the technical modeling of the **"Refill / Exchange"** scenario, which involved producing a full **Use Case Description, Activity Diagram, and Sequence Diagram**. I contributed to reviewing the **Feedback and Refund Modeling**, participated in actor identification, and helped polish the Deployment Diagram. This course challenged me to think beyond syntax and implementation, and instead focus on design clarity, system flow, and collaborative feedback. I leave this experience with deeper respect for planning, validation, and the critical role of diagrams in system engineering.

Programmer / Tester: Mohmmad ALabed

As the **Programmer and Tester**, I focused on implementing the logic of system behaviors from a modeling perspective. I led the modeling of the **"Return / Refund"** scenario, including its **Use Case, Activity, and Sequence Diagrams**, and took part in the development of the **Component and Deployment diagrams**. I also supported review efforts on the Class Diagrams and helped ensure that testable flows like error handling and refund processes were accurately captured. Working on this project gave me a practical understanding of how models become

blueprints for working systems, and I gained confidence in identifying dependencies, modeling edge cases, and ensuring logical consistency. Despite a heavy semester load and harsh external conditions, this experience sharpened my system design mindset.

2 Chapter 2: Requirement Elicitation, Analysis and Modeling

2.1 Requirement Statement / Business Description

(Task: Requirement Statement – Lead: Ahmad Elayyan, Contributors: Mohammad Obiad – content editing and formatting, Abdallah AboShoaib – technical review, Mohmmad ALabed – services validation)

Business Description

The **Online Water Shop** is a web-based delivery platform that enables customers to place, track, and manage their drinking water orders. Whether they are buying water bottles, requesting refills, or subscribing to weekly deliveries, the platform offers a convenient and reliable interface for interaction. The system allows customers to browse available water packages, add products to their cart, select delivery options, and complete the checkout process using secure payment methods. Our platform aims to streamline water distribution operations and provide efficient customer service, especially in residential and small-office environments where consistent access to clean drinking water is essential.

Business Services

The Online Water Shop provides the following services to ensure a convenient and effective water ordering process:

- **Online Ordering:** Customers can place single or bulk orders for water bottles.
- **Refill & Exchange:** Customers can return empty bottles and receive filled replacements.
- **Recurring Subscription Plans:** Customers can schedule automatic deliveries weekly or monthly.
- **Refund and Return Requests:** In cases of delivery damage or dissatisfaction, users can request a refund or exchange.
- **Real-time Tracking:** Customers receive live updates on order status and delivery progress.
- **Secure Payment Gateway:** Payments can be completed via cash on delivery, credit/debit cards, or local payment methods.
- **Customer Feedback:** Customers can rate services and provide feedback for continuous improvement.
- **Support Chatbot and Help Center:** Built-in support system for addressing customer inquiries.

Working Process

The business flow of handling customer requests in the Online Water Shop follows this structure:

1. **Customer Registration and Login:** New customers register with personal information and location data.
2. **Product Browsing:** Users view available water types and quantities.
3. **Order Placement:** Customers select water packages, specify quantity, choose delivery time, and confirm location.
4. **Payment Selection:** The system supports cash on delivery and digital payments through integrated gateways.
5. **Order Confirmation:** A unique order ID is generated and confirmation is sent via SMS/email.
6. **Processing and Dispatching:** Admin assigns the delivery task, and the warehouse dispatches bottles accordingly.
7. **Delivery:** The delivery team fulfills the order as scheduled and collects empty bottles if applicable.
8. **Post-Delivery Feedback / Refund (if needed):** If issues arise, the customer can submit a support ticket or initiate a refund request.

Company Capabilities

The Online Water Shop operates with a capacity to serve **200–300 active customers per week**, handling up to **1000 orders per month**. The system supports approximately **25–30 concurrent users**, with peak activity during morning hours. It integrates route optimization algorithms and inventory management to minimize delivery delays and product shortages.

We currently offer **3 main product categories**:

- **New Bottled Water Orders:** Standard and large sizes (5L, 10L, 20L).
- **Refill/Exchange Requests:** Customer returns empty bottles in exchange for filled ones.
- **Subscription Packages:** Weekly/monthly delivery plans for homes and businesses.

Workforce and Roles

To ensure operational efficiency, the Online Water Shop is managed by a small but structured team:

- **(2) Warehouse Staff:** Responsible for preparing and loading orders.
- **(3) Delivery Drivers:** Handle timely delivery and collect returns.
- **(1) Customer Service Agent:** Resolves inquiries, complaints, and refund requests.

- **(1) Administrator:** Manages inventory, assigns delivery tasks, and approves customer requests.
- **(1) System Operator / Manager:** Oversees platform performance, service quality, and business growth.

Business Vision

Our mission is to provide a seamless water delivery experience that fits the needs of modern urban lifestyles. Through automation, scheduling, and customer-centered design, the system saves time for both customers and staff. The long-term goal is to expand the platform to support partnerships with other local water vendors and smart routing systems, enabling broader geographic coverage and optimized distribution.

2.2 USER and SYSTEM Requirements

USER REQUIREMENTS [Group Task] [MODIFIED]

Lead: Ahmad Elayyan
Contributors: Abdallah AboShoaib (UML coordination), Mohammad Obiad (API + flow), Mohmmad ALabed (UX logic)

- **UR1:** Customers shall be able to register a new account securely.
- **UR2:** Customers shall be able to log in using their phone number and password.
- **UR3:** Customers shall be able to log out from their account securely.
- **UR4:** Customers shall place water orders by selecting bottle type and quantity.
- **UR5:** Customers shall schedule delivery slots (2-hour windows).
- **UR6:** Customers shall request bottle refill or exchange service.
- **UR7:** Customers shall use a wizard to estimate monthly water consumption based on household size and average usage.
- **UR8:** Customers shall report delivery issues or product defects.
- **UR9:** Customers shall request product returns or refunds.
- **UR10:** Customers shall provide feedback (rating + comments) after delivery.
- **UR11:** Customers shall view their accumulated loyalty points.
- **UR12:** Customers shall redeem loyalty points during checkout.
- **UR13:** Customers shall receive SMS/email notifications for order confirmations, delivery updates, and feedback requests.
- **UR14:** Staff shall manage customer orders and delivery routes.
- **UR15:** Staff shall manage customer feedback and configure system settings.

SYSTEM REQUIREMENTS [Group Task][MODIFIED]

Lead: Abdallah AboShoaib
Contributors: Ahmad Elayyan (reviewing), Mohammad Obiad (discussion), Mohmmad ALabed (discussion)

- **UR1:** The system shall enable users to create personal accounts by providing their phone number, full name, password, and address.
  - **SR1.1:** The system shall allow the user to enter the following fields: full name, phone number, password, and delivery address.
  - **SR1.2:** The system shall validate that all fields are complete and correctly formatted.

  - **SR1.3:** Passwords shall contain at least 8 characters and one uppercase letter.
  - **SR1.4:** Duplicate phone numbers shall not be accepted.
  - **SR1.5:** A success message shall be displayed once the account is created.

- **UR2:** The system shall allow users to log in using their phone number and password.
  - **SR2.1:** Invalid login attempts shall display an appropriate error message.
  - **SR2.2:** Passwords shall be hashed using SHA256 before storage.
  - **SR2.3:** Accounts shall be locked after 5 failed login attempts.
  - **SR2.4:** A ”Remember Me” checkbox shall store the user’s login session for 7 days unless manually logged out.

- **UR3:** The system shall allow users to place water orders by selecting bottle type, quantity, and delivery time.
  - **SR3.1:** Orders shall include: product type, quantity, delivery window, and location.
  - **SR3.2:** The system shall validate stock availability and delivery slot.
  - **SR3.3:** A confirmation message with order ID shall be sent via SMS.

- **UR4:** The system shall support refill/exchange requests for empty water bottles.
  - **SR4.1:** Refill option shall be part of the order form.
  - **SR4.2:** On delivery, system shall register quantity of empty bottles returned.

- **UR5:** The system shall allow customers to subscribe to recurring deliveries (weekly or monthly).
  - **SR5.1:** Subscription info shall include interval, quantity, bottle type, and start date.
  - **SR5.2:** Scheduler shall auto-generate orders from subscriptions daily at 00:00.

- **UR6:** The system shall allow users to estimate water consumption.
  - **SR6.1:** Wizard shall calculate: occupants × 3.5L × 30 days.
  - **SR6.2:** Results shall appear within 0.5 seconds and pre-fill order form if accepted.

- **UR7:** The system shall allow return/refund requests for damaged or missing products.
  - **SR7.1:** Refund form shall allow entering reason and attaching up to 3 images (5MB).
  - **SR7.2:** Staff dashboard shall sort open refund tickets by submission time, showing the oldest first.

- **UR8:** The system shall collect feedback after delivery.
  - **SR8.1:** Feedback form shall include rating (1–5) and optional comment.
  - **SR8.2:** Feedback link expires 5 days after delivery.

- **UR9:** The system shall manage loyalty points.

  - **SR9.1:** 1 point is earned for each ILS 10 spent.
  - **SR9.2:** Points can be redeemed in blocks of 10 at checkout.

- **UR10:** The system shall notify users via SMS/email about order status and feedback links.
  - **SR10.1:** Twilio API shall send SMS delivery confirmations. SendGrid API shall send email updates for order status and feedback requests.
  - **SR10.2:** All notifications shall be timestamped and logged.

- **UR11:** Admin/staff shall be able to manage customer orders and delivery routes.
  - **SR11.1:** Admin dashboard shall allow filtering of customer orders by date and status.
  - **SR11.2:** Admin dashboard shall allow refund requests to be approved or rejected.
  - **SR11.3:** Admin dashboard shall allow pausing of active delivery subscriptions.
  - **SR11.4:** The system shall update the admin dashboard in real-time using WebSocket, or fallback to auto-refresh every 30 seconds.

- **UR12:** Admin/staff shall be able to manage customer feedback and configure system settings.

2.3 SCENARIOs

Scenario S-1: Place Standard Order (Author: Abdallah AboShoaib) [MODIFIED]

Initial Assumption

The customer has at least one item in the cart. Cart quantities do not exceed the available stock. Payment methods are operational and integrated with the system.

Normal Flow

1. Customer clicks the **"Checkout"** button.
2. System verifies stock levels and calculates total amount.
3. System displays order summary with all item details and pricing.
4. Customer selects a payment method (e.g., credit card, PayPal).
5. Payment Gateway authorizes the transaction.
6. System creates a new order record with status = NEW and reserves stock.
7. Confirmation is sent via SMS and email within 60 seconds.

Alternative Flow 1: Guest Checkout

If the customer is not logged in, the system allows checkout as a guest. The UI optionally prompts the user to create an account but does not block the process.

Alternative Flow 2: Promo Code Applied

If the user applies a valid promo code before payment, the system validates it, applies the discount, and updates the total price.

Error Flow 1: Out-of-Stock Items

If the stock validation fails at step 2, the system returns an HTTP 409 status, highlights affected items, and redirects the user to the cart for review.

Error Flow 2: Payment Declined

If the payment is declined, the system displays a *"Payment Declined"* message and allows the user to retry or choose a different payment method.

System State on Completion

- New order is stored in the database with reserved inventory.
- Notifications are dispatched within 60 seconds.
- External API calls must return within 2 seconds to avoid blocking the process.

Scenario S-2: Schedule Delivery Window (Author: Abdallah AboShoaib) [MODIFIED]

Initial Assumption

The customer has already placed an order (status = NEW). The delivery address has been successfully geocoded and is valid.

Normal Flow

1. Customer clicks the **"Schedule Delivery"** option.
2. System shows calendar starting 24 hours from current time with 2-hour time slots.
3. Customer selects a delivery slot (e.g., "22 May 2025, 14:00–16:00").
4. System checks slot availability and reserves it.
5. Delivery details are updated in the order and sent to the Route Optimizer service.

Alternative Flow: ASAP Suggestion

If no delivery slot is available in the next 48 hours, the system highlights the earliest available future slot.

Error Flow: Address Outside Delivery Area

If the address is outside the delivery zone, the UI displays: *"Address not in delivery zone"* and blocks confirmation.

System State on Completion

- Delivery slot is stored in the order record.
- Route Optimizer receives the updated route.
- Geocoder and scheduler APIs are expected to respond within 2 seconds.

Scenario S-3: Bulk Order with Tiered Discount (Author: Ahmad Elayyan) [MODIFIED]

Initial Assumption

Customer is logged in and has at least 100 bottles in the cart. Pricing Service is operational and reachable.

Pre-Conditions

- Cart subtotal is at least 100 bottles.
- Pricing engine is online and fully functional.

Normal Flow

1. Customer navigates to the product catalog, searches for the desired bottled water size, selects the quantity of 120, and clicks the ”Update Cart” button to confirm the addition.
2. System detects the bulk order and calls the Pricing Service to apply discounts.
3. Pricing Service applies a 5% discount for qualifying bulk orders.
4. Cart and invoice are updated to reflect the discount and new total price.

Alternative Flow: Higher Tier Discount

If the customer increases the order quantity to 200 or more bottles, the system recalculates and applies a 10% discount, updating the cart and invoice immediately.

Error Flow: Pricing Service Failure

If the Pricing Service is unavailable or offline, the system displays a yellow banner stating: *“Bulk discount temporarily unavailable”*, disables the discount functionality for the user, and notifies the operations team for manual intervention.

System State on Completion

- Discount is applied and visible in both invoice and cart summary.
- Discount calculation must complete within 1 second to avoid checkout delay and ensure smooth user experience.

Scenario S-4: Refill / Exchange Empty Bottles (Author: Mohammad Obiad) [MODIFIED]

Initial Assumption

Customer is logged in and has eligible empty bottles (10L or 19L) recorded in the system. System time is synced and accurate.

Pre-Conditions

- Customer owns at least one eligible 10L or 19L empty bottle.
- Requested pickup is scheduled at least 24 hours in advance.

Normal Flow

1. Customer logs into the system, navigates to the Refill/Exchange page, and initiates a refill or exchange request.
2. Customer selects the quantity of empty bottles and preferred pickup date.
3. System validates customer's eligibility and availability of pickup slots, then creates a combined logistics job for pickup and delivery.
4. Driver receives the task and confirms it.
5. Once confirmed, customer is charged and inventory is updated accordingly.

Alternative Flow: Mixed Transaction

Customer adds new bottled water purchase in the same checkout session. The system merges both purchase and refill orders into one combined delivery job.

Error Flow: No Empties Found

If the driver reports zero empty bottles during pickup, the system charges the customer full price and notifies the Support Agent for follow-up.

System State on Completion

- The logistics job is logged and marked as completed after confirmation.
- Inventory reflects the returned empty bottles and delivered full bottles.
- The Logistics API responds within 2 seconds to ensure timely processing.

Scenario S-5: Return / Refund for Damaged Product (Author: Mohammad Alabed) [MODIFIED]

Initial Assumption

Customer is logged in and has a valid delivered order eligible for return or refund. User has valid photo evidence for the damaged product.

Pre-Conditions

- Order status is DELIVERED and less than 48 hours have passed since delivery.
- Uploaded files must be in JPEG or PNG format, with a maximum size of 5MB per image.

Normal Flow

1. Customer navigates to the return/refund page, initiates a return or refund request, uploads up to 3 photos as evidence, and selects the desired resolution (refund or replacement).
2. Support Agent reviews the request within 2 business days.
3. System updates the customer with the decision and issues a refund or replacement accordingly.

Alternative Flow: Partial Refund

Support Agent approves refund for some items; loyalty points are recalculated accordingly.

Error Flow 1: Expired Return Window

If the return/refund request is submitted after 48 hours from delivery, the system displays: *"Return window expired."* and blocks the submission.

Error Flow 2: Invalid Upload

If uploaded files exceed 5MB or are in an unsupported format, the system shows: *"Invalid file type or size."*

System State on Completion

- Return record is stored with detailed resolution information.
- Customer account and order history reflect the updated status.
- File validation occurs client-side before submission to ensure compliance.

2.4 Effort/Time Estimation Calculation [MODIFIED]

Lead: Ahmad Elayyan; contributors: Abdallah AboShoaib (estimation logic), Mohammad Obiad (reviewing), Mohmmad ALabed (discussion)

UR	Dev Count	Effort (pw)	Total Effort
UR1: Account Registration	2	2pw	4pw
UR2: Login Security	2	1pw	2pw
UR3: Standard Order Placement	2	3pw	6pw
UR4: Bottle Exchange	2	2pw	4pw
UR5: Subscription Delivery	1	2pw	2pw
UR6: Water Consumption Wizard	1	1pw	1pw
UR7: Return/Refund	1	1pw	1pw
UR8: Feedback System	1	1pw	1pw
UR9: Loyalty Points	1	1pw	1pw
UR10: Notifications (SMS/Email)	1	1pw	1pw
UR11: Admin Dashboard	2	2pw	4pw
Total	-	**17pw**	**27pw**

Table 1: Effort Estimation per Requirement

Assumptions: 4 developers are available. Each person works full-time (1 person-week = 1 week effort).

1. Time Calculation

- Basic Time (no buffer): $\frac{27 \text{ pw}}{4 \text{ people}} = 6.75$ weeks
- Estimated Project Time (with 30% buffer): $6.75 \times 1.3 = 8.78 \approx 9$ weeks

2. Cost Estimation

- Weekly salary per developer = 330 USD
- Team weekly cost = $330 \times 4 = 1320$
- Base cost = $1320 \times 9 = 11,880$

3. Profit Estimation

- **Minimum profit (12%)** = $11,880 \times 1.12 = 13,305.6$
- **Maximum profit (30%)** = $11,880 \times 1.3 = 15,444$

Conclusion

- Project cost range: **$13,305 – $15,444**
- Recommended timeline: **9 weeks**
- Team: 4 developers

2.5 Actor Analysis [MODIFIED]

Task: Actor Analysis – Lead: Ahmad Elayyan
Contributors: Abdallah AboShoaib (reviewing), Mohammad Obiad (clarity feedback), Mohmmad ALabed (formatting)

The following table identifies and describes the main actors that interact with the **Online Water Shop** system. These actors are categorized into internal human users and external system components.

Actor	Description
Customer	A registered user who browses products, places orders, schedules deliveries, subscribes to weekly/monthly services, provides feedback, and can request refunds or exchanges.
Guest User	An unauthenticated user who is allowed to place one-time orders without registering. May optionally create an account after checkout.
Warehouse Staff	Internal employee responsible for managing stock, packing orders, handling refills, and updating the inventory after processing.
Delivery Driver	A field user who receives delivery/pickup assignments, confirms task completion, and reports bottle returns or exceptions.
Support Agent	Handles customer service tasks including reviewing refund/return requests, managing tickets, and resolving user complaints within SLA.
Administrator	A privileged user who manages product catalog, pricing tiers, delivery settings, user permissions, and oversees the entire platform.
Payment Gateway (external)	Third-party API service (e.g., PayPal, Visa/MasterCard) used for securely processing online payments.
Pricing Service (external)	External microservice responsible for calculating discounts during bulk orders, based on quantity thresholds.
Route Optimizer (external)	Automated service that calculates optimal delivery paths using real-time and scheduled orders, improving delivery efficiency.
SMS/Email Notification Service (external)	Integrated communication service responsible for sending order confirmations, delivery updates, feedback requests, and return notifications.

2.6 USE-CASE Diagram [MODIFIED]

Lead: Mohammad Obiad; Contributors: Ahmad Elayyan, Abdallah AboShoaib, Mohmmad ALabed (Review, Edit)

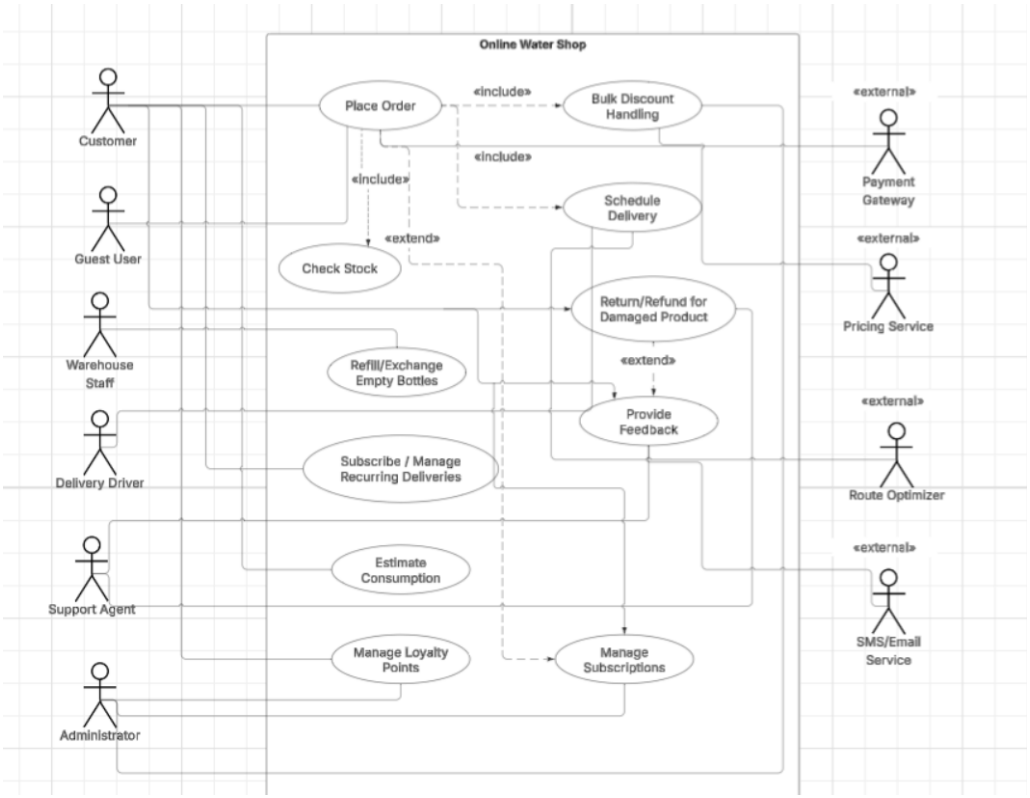


Figure 6: Overall Use-Case Diagram of the Online Water Shop

2.7 Description of Key Use-Cases

Ahmad Elayyan's Use Case Description: Estimate Monthly Consumption [MODIFIED]

System Name	Online Water Shop
Use Case Title	Estimate Monthly Consumption
Description	This use case allows a registered customer to estimate their household's monthly water consumption through a short interactive wizard. The system processes basic household information and returns a suggested number of bottles per month based on the input.
Primary Actor	Customer
Supporting Actors	Analytics Engine – processes user inputs to calculate estimated consumption using predefined usage patterns.
Initial Assumptions	– User is logged in and has not used the wizard in the past 24 hours. – Device/browser supports JavaScript. – To prevent redundant calculations and reduce server load, wizard usage is limited to once every 24 hours per user.
Trigger	Customer clicks the "Estimate My Usage" button on the homepage.
Pre-Conditions	– User must complete the 5-step form within 3 minutes. – All input fields must be valid (e.g., numeric occupancy values).
Main Flow	1. System presents question 1: number of people in the household. 2. System presents question 2: average water use per person. 3. System presents optional questions about cooking habits, pets, and other relevant factors. 4. System calculates the total estimated monthly water requirement. 5. User receives a suggestion, e.g., 90 liters/month, which equals 6×15L bottles.

Alternative Flow A1 – Incomplete Wizard	– If the user closes the wizard midway, no result is calculated. – The wizard resets on the user's next visit.
Exception Flow E1 – Input Validation Error	– If any numeric field is invalid, the system displays an error message. – The flow pauses until the user corrects the input.
Post-Conditions	– The result is cached for 24 hours to optimize performance. – Analytics logs include response time and input details for monitoring.
System State on Completion	– The estimation summary is stored temporarily. – The user is redirected to the "Add to Cart" page with the suggested bottle quantities.

Abdallah AboShoaib's Use Case Description: Place Standard Order [MODIFIED]

System Name	Online Water Shop
Use Case Title	Place Standard Order
Description	Customer places an order for water bottles using the app or website. This includes selecting bottle size, quantity, payment method, and confirming delivery address. The system verifies product availability and processes the payment.
Primary Actor	Customer
Supporting Actors	Inventory System, Payment Gateway, SMS Service
Initial Assumptions	– User is authenticated or using guest checkout mode. – Shopping cart contains at least one valid item.
Trigger	Customer presses the "Checkout" button.
Pre-Conditions	– Valid payment methods are available (e.g., PayPal, Visa). – Stock is available for all selected products. – User session is active and valid.

Main Flow	1. System checks stock availability for items in the cart. 2. System displays an order summary including selected items, quantities, and total price. 3. Customer selects a payment method and enters payment details. 4. Payment Gateway authorizes the transaction successfully. 5. System creates a new order record with status set to NEW and reserves stock. 6. Confirmation of the order is sent to the customer via SMS and email.
Alternative Flow A1 – Guest Checkout	– If the customer is not logged in, the system prompts to register but allows continuing as a guest. – Guest orders do not save user data for future use unless the guest opts to create an account.
Error Flow E1 – Payment Failed	– If the payment is declined, the system halts the process and displays "Payment failed, please try again". – Customer may retry payment or choose a different payment method.
Post-Conditions	– New order is stored in the system database. – Stock quantities are reserved accordingly. – Confirmation notifications are sent to the customer.
System State on Completion	– Order status is NEW. – Order is linked to the user's account history if logged in.

Abdallah AboShoaib's Use Case Description: Schedule Delivery Window [MODIFIED]

System Name	Online Water Shop
Use Case Title	Schedule Delivery Window

Description	This use case enables a customer to select a preferred delivery window after checkout. The system validates time slots and confirms availability before scheduling.
Primary Actor	Customer
Supporting Actors	Scheduler, Driver, Notification System
Initial Assumptions	– Order status is NEW. – Delivery zones and time slots are pre-defined.
Trigger	User selects delivery time after order confirmation.
Pre-Conditions	– Delivery window must be within working hours. – Same-day delivery only available before 12 PM.
Main Flow	1. System displays available time slots. 2. Customer selects preferred window. 3. System checks for conflicts. 4. Driver assigned to slot. 5. Notification sent to customer.
Alternative Flow – Reschedule	– Customer changes slot → System revalidates. – Old assignment is released.
Error Flow – Slot Full	– If no drivers are available, system shows ”Fully Booked”.
Post-Conditions	– Slot reserved. – Job appears on driver’s schedule.
System State on Completion	– Order marked as “SCHEDULED”.

Mohammad Obiad’s Use Case Description: Refill / Exchange Empty Bottles [MODIFIED]

System Name	Online Water Shop
Use Case Title	Refill / Exchange Empty Bottles

Description	This use case describes how customers schedule pickups for empty bottles and request refills or exchanges. Logistics integration allows a combined pickup-delivery job to be generated automatically.
Primary Actor	Customer
Supporting Actors	Warehouse Staff, Delivery Driver, Logistics System
Initial Assumptions	– Customer has empty 10L or 19L bottles linked to their account. – System time is accurate to coordinate date checks.
Trigger	Customer selects ”Refill/Exchange Bottles” from the dashboard.
Pre-Conditions	– Customer has 1 eligible empty bottles. – Pickup request is 24 hours in advance.
Main Flow	1. Customer enters number of empties and selects refill or exchange. 2. Customer picks desired pickup date. 3. System validates date and bottle quantity. 4. System creates logistics task (combined pickup + delivery). 5. Driver receives task details. 6. Inventory updated upon pickup confirmation.
Alternative Flow A1 – Mixed Transaction	– If customer adds new bottle purchases: ∗ System merges refill and purchase into a single delivery.
Exception Flow E1 – No Empties at Pickup	– Driver reports zero empties found. – System charges full price for bottles. – Support Agent is notified.
Post-Conditions	– Refill/exchange job marked completed. – Inventory reflects returned/issued bottles.

System State on Completion	– Inventory updated. – Delivery job archived.

Mohmmad ALabed's Use Case Description: Return / Refund for Damaged Product [MODIFIED]

System Name	Online Water Shop
Use Case Title	Return / Refund for Damaged Product
Description	Customers can request a return or refund in case of a damaged or defective product. The process includes photo upload, manual review by the support team, and automatic refund/replacement issuance.
Primary Actor	Customer
Supporting Actors	Support Agent, SMS/Email Service, Finance System
Initial Assumptions	– Customer has a delivered order 48 hours ago. – Product was reported damaged.
Trigger	Customer clicks on "Return / Refund" from order history.
Pre-Conditions	– Order has status = DELIVERED. – Upload limit: max 3 images (5MB each).
Main Flow	1. Customer selects return option and attaches up to 3 images. 2. System validates format and file size. 3. Ticket is created and assigned to Support Agent. 4. Agent reviews request within 2 business days. 5. If approved, system issues refund or replacement. 6. Customer notified of resolution via SMS/email.
Alternative Flow A1 – Partial Refund	– Agent approves refund for subset of items. – Loyalty points adjusted accordingly.
Exception Flow E1 – Expired Return Window	– Customer attempts return after 48 hours. – System blocks submission and shows warning.

Exception Flow E2 – Invalid Image Format	– Uploaded file fails format/size check. – Error message shown: “Invalid file type or size.”
Post-Conditions	– Ticket is stored with resolution status. – Customer account updated.
System State on Completion	– Refund issued or replacement queued. – Return history updated.

2.8 ACTIVITY Diagram [MODIFIED]

Team Lead: Ahmad Elayyan.
Contributing Members: Abdallah AboShoaib (Logic Review), Mohammad Obiad (Flow Mapping), Mohmmad ALabed (Validation and Final Checks)

This activity diagram presents the high-level business process for placing and fulfilling water orders through the Online Water Shop platform. It covers product selection, bulk pricing logic, checkout, payment handling, stock verification, scheduling, delivery, and post-delivery services such as feedback, refill/exchange, and refund. The diagram uses swim-lanes to represent interactions between different system actors and external services.

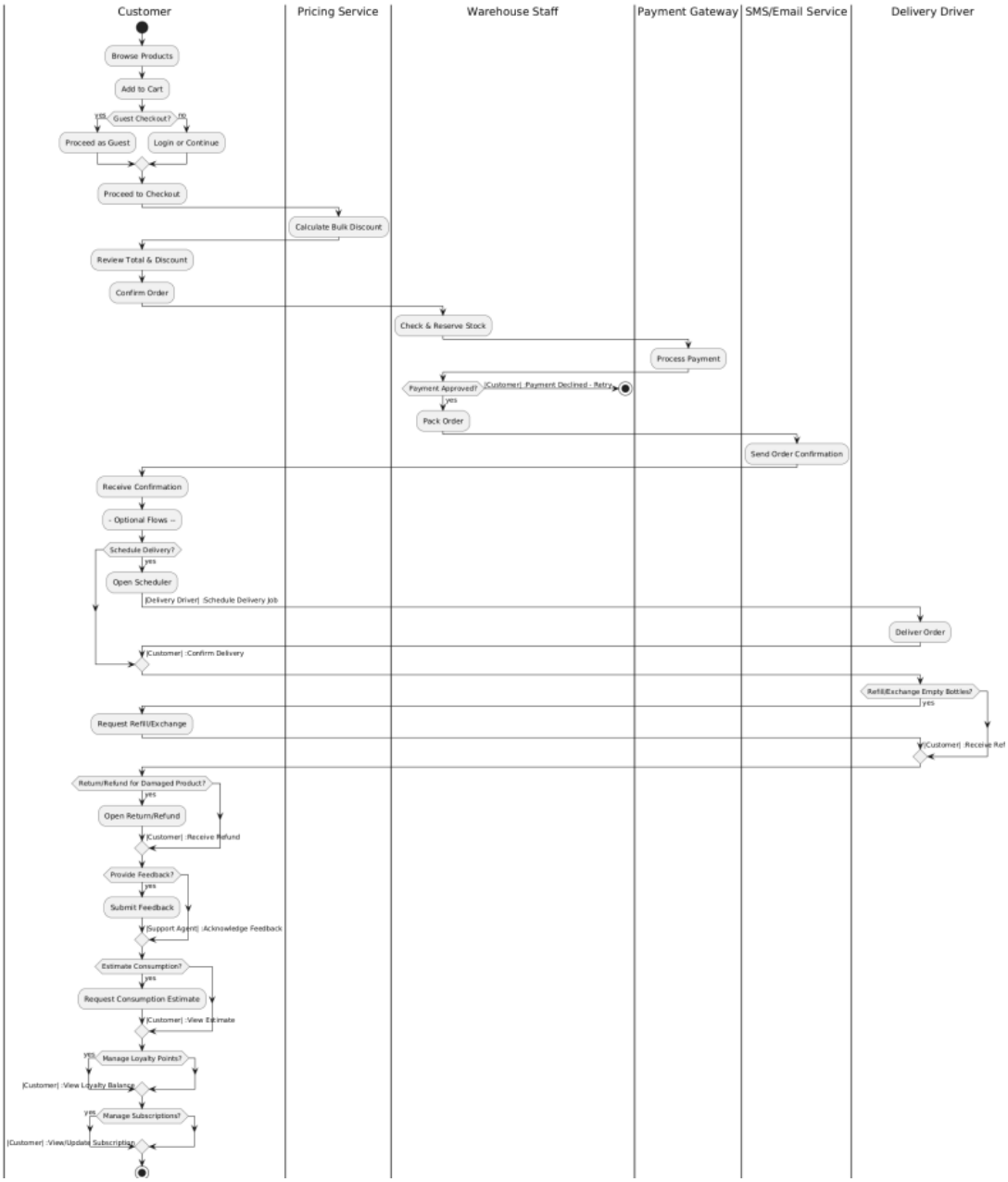


Figure 7: Overall Business Activity Diagram – Online Water Shop

2.9 Instance Activity Diagrams

Each instance activity diagram illustrates a specific system behavior related to a key use-case implemented by a team member. These diagrams highlight normal flows, decision points, error handling, and interactions across actors using swim-lanes. Together, they validate the logical execution paths defined in Section 2.7.

Abdallah AboShoaib's Diagram: Place Standard Order [MODIFIED]

This diagram captures the full order placement process. It begins with the customer initiating checkout, followed by inventory validation (SR2.2), payment authorization (SR1.1), and ends with order creation and confirmation. Alternative and error flows (e.g., out-of-stock, payment declined) are embedded as conditional branches.

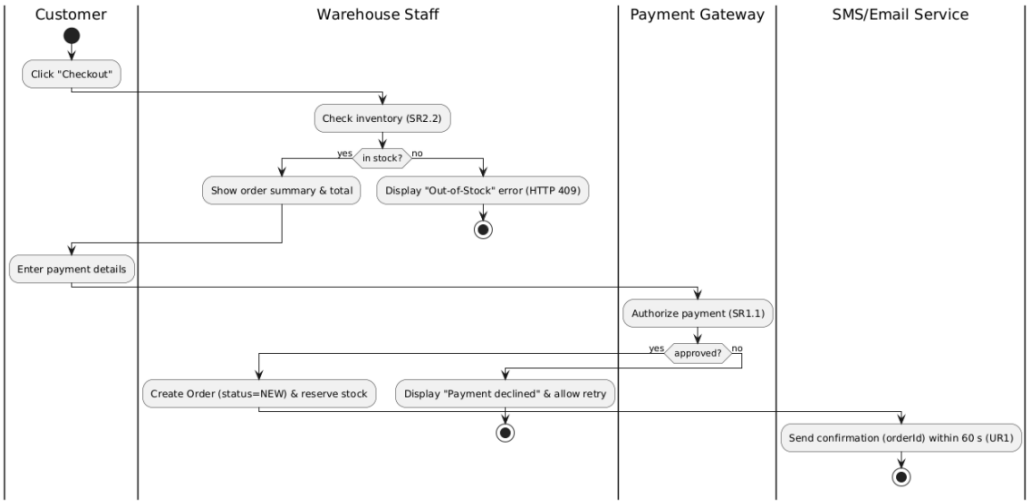


Figure 8: Instance Activity Diagram — Place Order (Abdallah AboShoaib)

Mohammad Obiad's Diagram: Refill / Exchange Empty Bottles [MODIFIED]

This activity represents the customer's ability to request bottle refills or exchanges. The process checks whether the pickup date is 24 hours in advance, then creates a delivery job accordingly. Warehouse staff and drivers are integrated through joint actions. Validation logic (e.g., date check, SR5.2) is included.

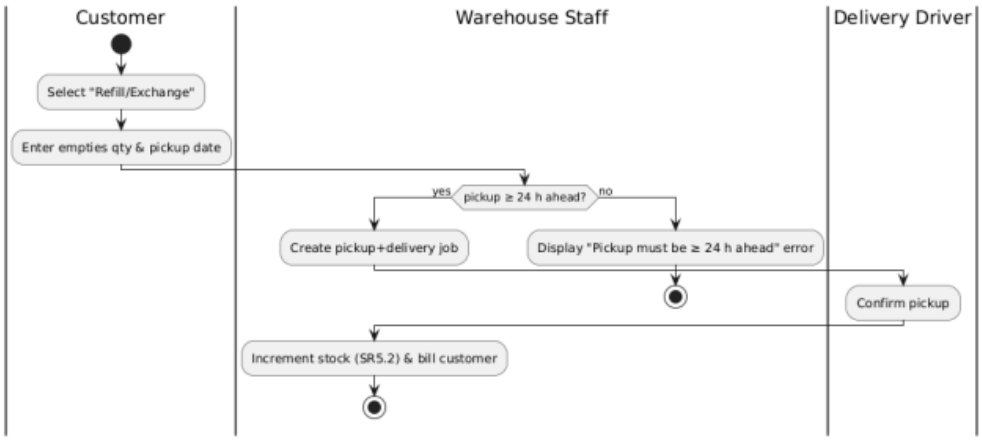


Figure 9: Instance Activity Diagram — Refill / Exchange Bottles (Mohammad Obiad)

Ahmad Elayyan's Diagram: Bulk Order with Tiered Discount [MODIFIED]

This diagram explains the dynamic pricing flow for bulk water purchases. When cart quantities exceed predefined thresholds, the Pricing Service is invoked to calculate tiered discounts. This logic is embedded before checkout, and the discount is reflected in the final invoice.

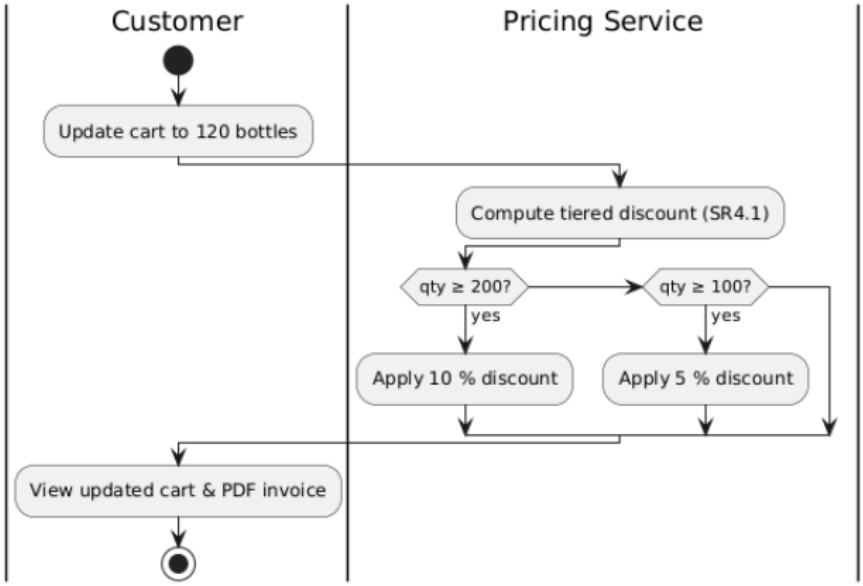


Figure 10: Instance Activity Diagram — Bulk Discount Handling (Ahmad Elayyan)

Mohmmad ALabed's Diagram: Return / Refund for Damaged Product [MODIFIED]

This instance diagram models the return/refund use-case. A customer uploads evidence, triggering a review process by the support agent. Based on evaluation, a refund or replacement is issued. The diagram includes conditions for expired return windows and invalid uploads.

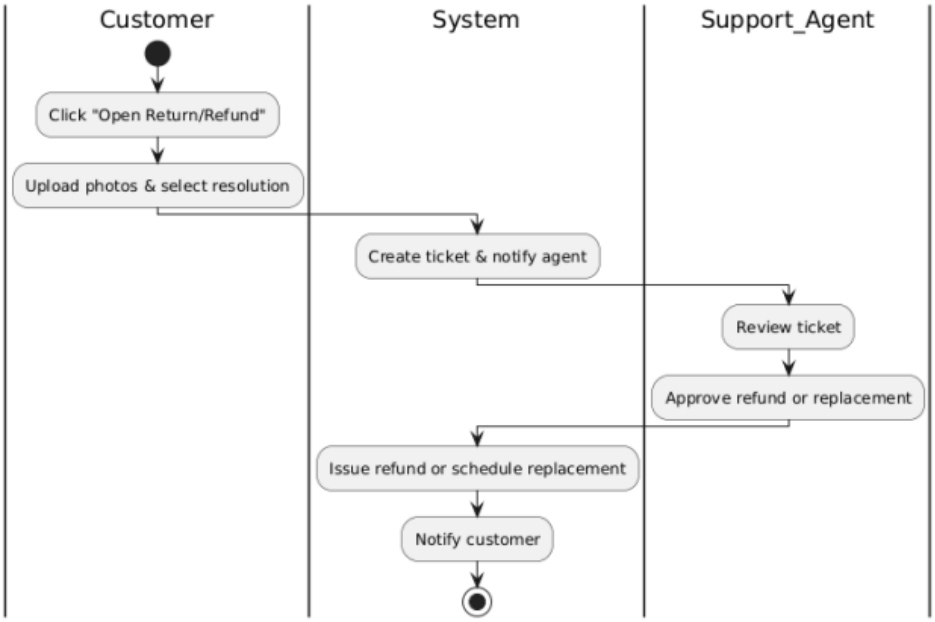


Figure 11: Instance Activity Diagram — Return / Refund Request (Mohmmad ALabed)

Abdallah AboShoaib's Diagram: Schedule Delivery Window [MODIFIED]

This activity diagram focuses on the post-order process of selecting a delivery window. It highlights customer interaction with the scheduling calendar, slot availability logic, and how the system either reserves the selected slot or suggests the next best option using the Route Optimizer.

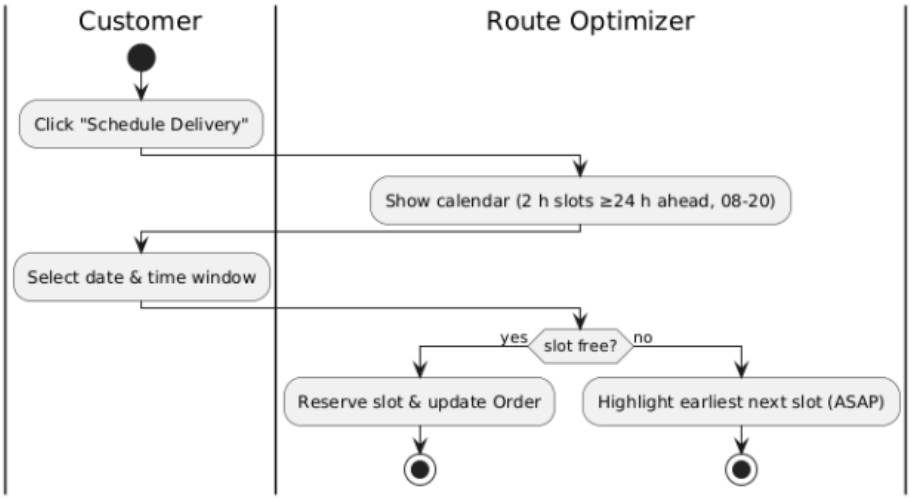


Figure 12: Instance Activity Diagram — Schedule Delivery (Abdallah AboShoaib)

3 Chapter 3: System Analysis and Modelling

3.1 3.1 System CLASS Diagrams [MODIFIED]

Team Lead: Mohammad Obiad
Contributing Members: Ahmad Elayyan (Attribute Relationship Review), Abdallah AboShoaib (Service Layer Validation), Mohmmad ALabed (Object Operations Methods)

The following section presents the structural analysis of the Online Water Shop system. The class models are divided into two views:

- The **Analysis Class Diagram** outlines the core business entities and relationships from a conceptual standpoint.
- The **Detailed Class Diagram** expands the same model with data attributes, operations, and interaction with services.

3.1.1 Analysis Class Diagram

This diagram identifies the major conceptual entities in the system such as 'Customer', 'Order', 'Cart', and 'RefillRequest'. It also shows how these entities interact with external services (e.g., 'PaymentGateway', 'PricingService', 'RouteOptimizer') and internal

roles like ‘SupportAgent‘ and ‘DeliveryDriver‘. Dashed arrows represent service use or invocation.

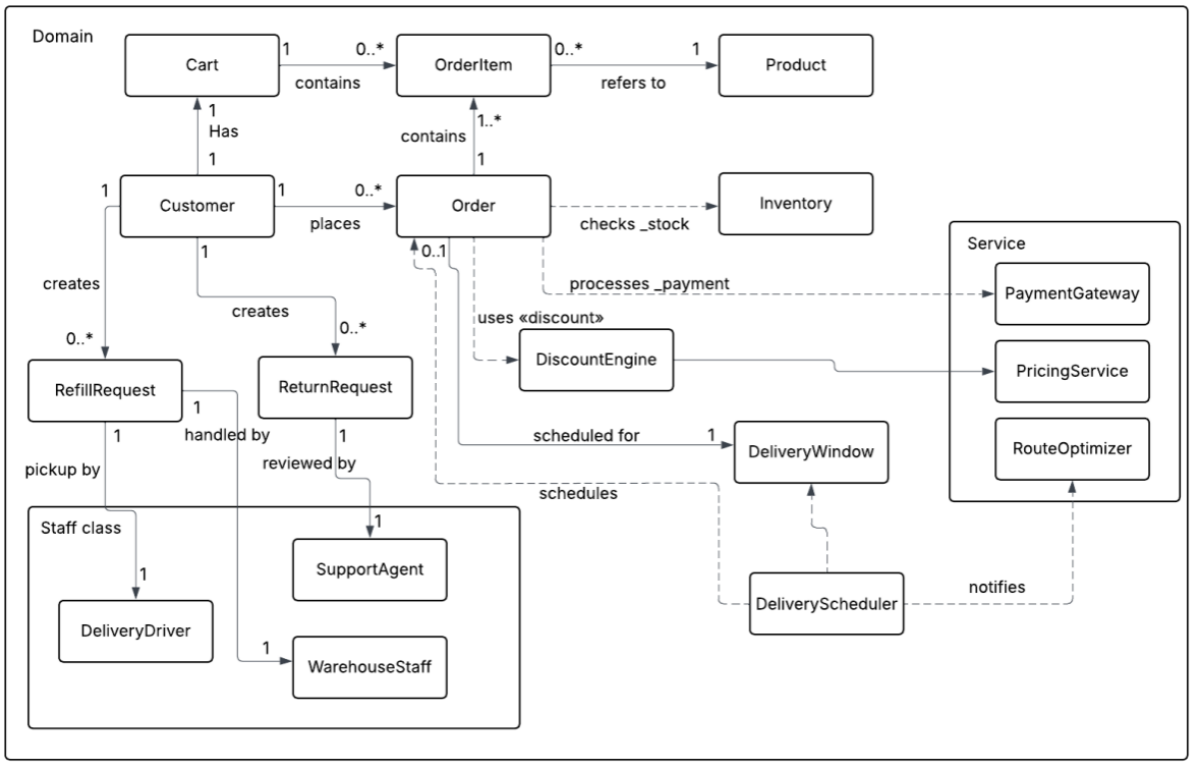


Figure 13: Analysis Class Diagram – Online Water Shop

3.1.2 Detailed Class Diagram [MODIFIED]

This diagram refines the previous one by specifying:

- Key attributes (e.g., ‘orderId‘, ‘productName‘, ‘status‘)
- Main operations (e.g., ‘placeOrder()‘, ‘authorizePayment()‘, ‘reserveStock()‘)
- Object composition and navigation logic
- Service integration methods (e.g., ‘computeRate()‘, ‘getDiscountRate()‘, ‘updateDeliveryJob()‘)

It enables a clear understanding of object-level responsibilities, inter-class messaging, and encapsulation of business logic. Each class is designed to uphold high cohesion and follows naming consistency based on the domain.

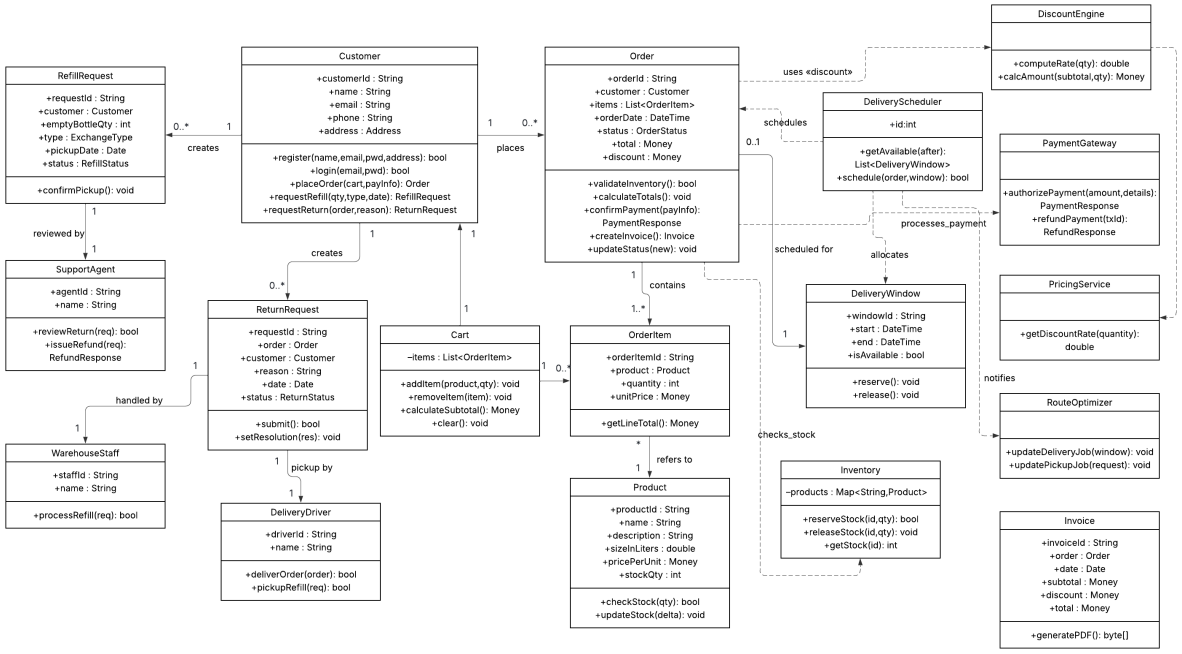


Figure 14: Detailed Class Diagram – Online Water Shop

3.2 SEQUENCE Diagrams

Each of the following sequence diagrams corresponds to a key use-case previously described in Section 2.7. These diagrams show the lifelines of involved actors and the flow of messages exchanged to fulfill each system operation.

Abdallah AboShoaib's Diagram: Place Standard Order [MODIFIED]

This diagram represents the interaction between a customer and the system while placing an order. It includes cart validation, payment authorization, order creation, and confirmation notifications.

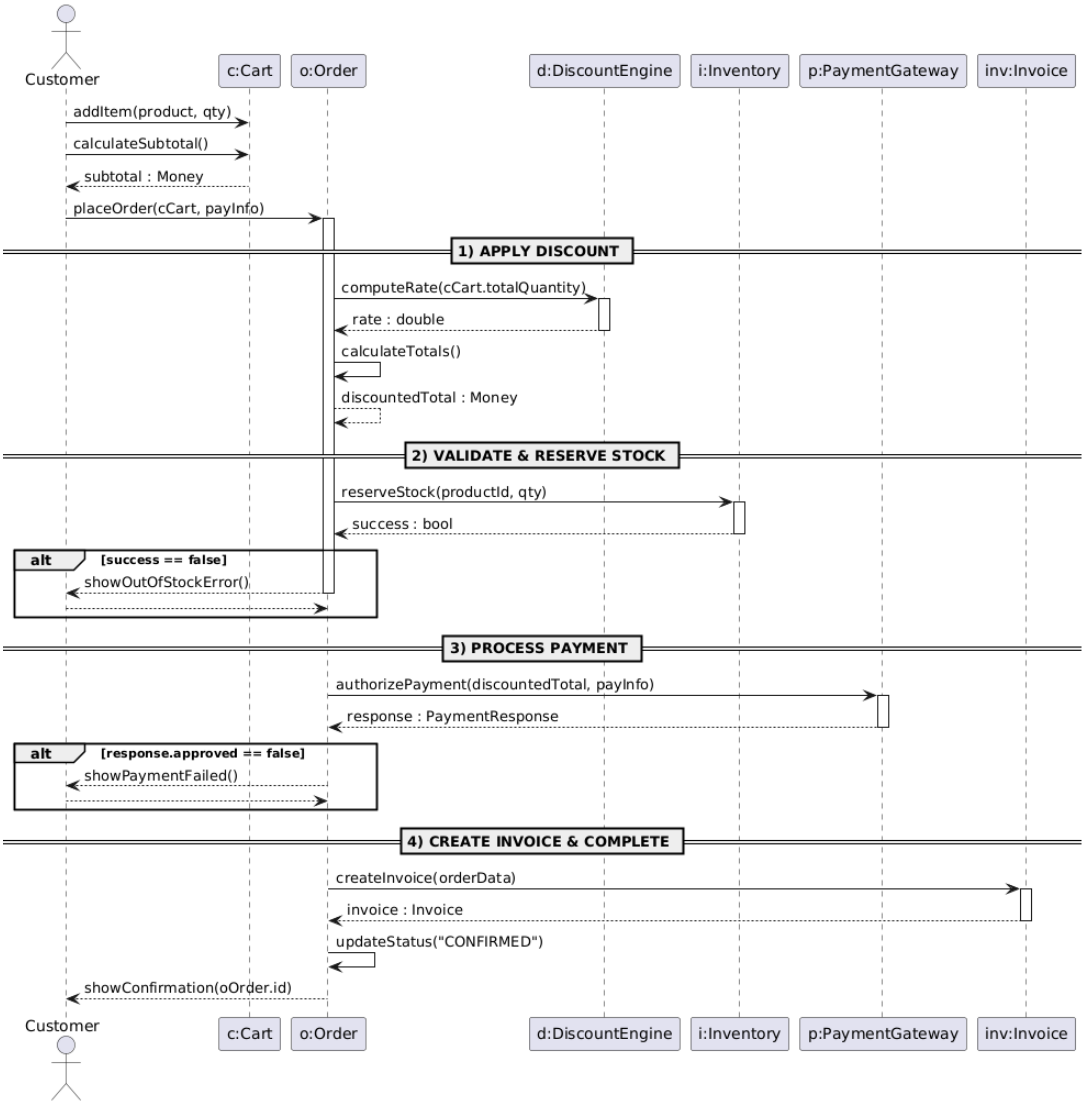


Figure 15: Sequence Diagram — Place Order (Abdallah AboShoaib)

Abdallah AboShoaib's Diagram: Schedule Delivery Window [MODIFIED]

This sequence illustrates how a customer selects and reserves a delivery time window. The system checks availability and notifies the route optimizer.

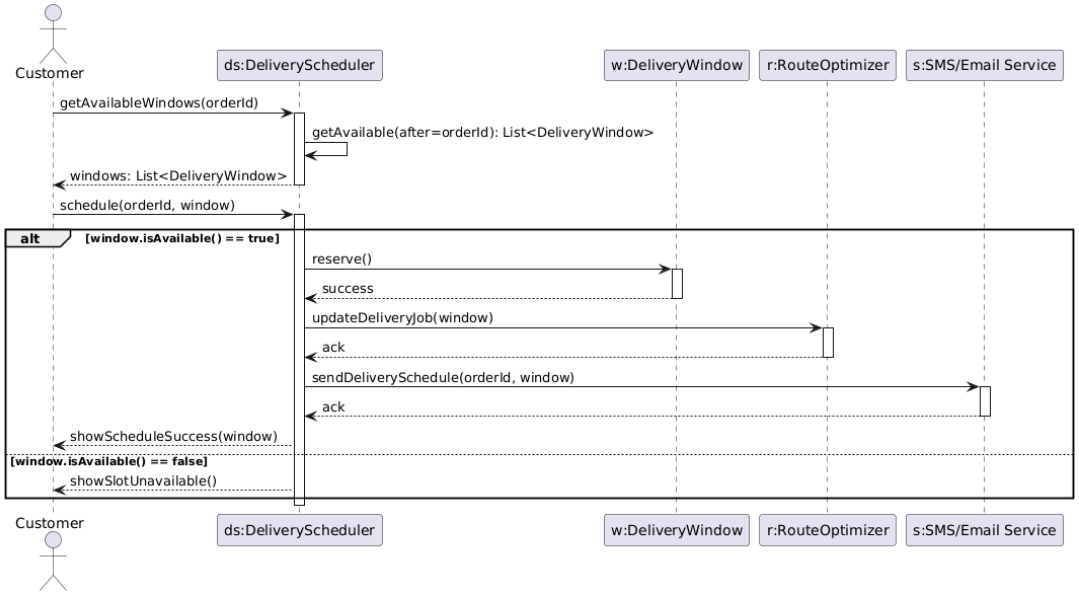


Figure 16: Sequence Diagram — Schedule Delivery (Abdallah AboShoaib)

Ahmad Elayyan's Diagram: Bulk Order with Tiered Discount [MODIFIED]

This diagram shows the flow where a customer updates cart quantities, triggering the Pricing Service to apply a tiered discount, and the invoice updates accordingly.

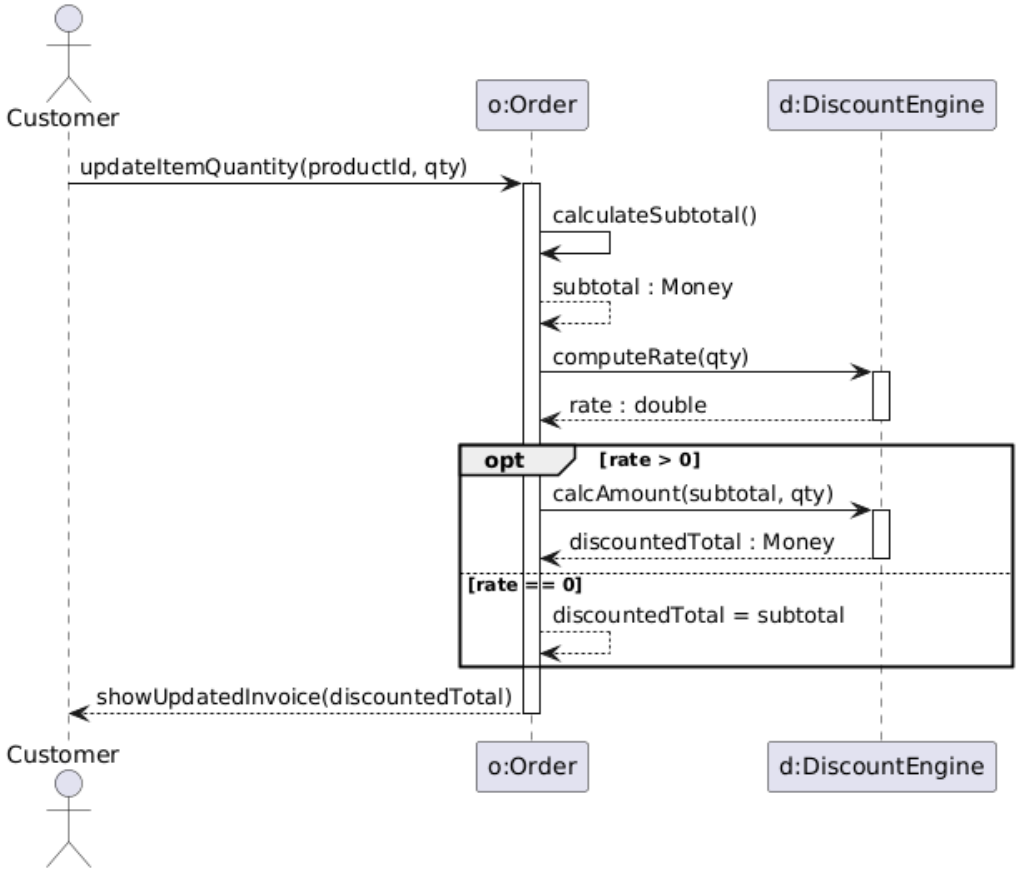


Figure 17: Sequence Diagram — Bulk Order with Discount (Ahmad Elayyan)

Mohammad Obiad's Diagram: Refill / Exchange Empty Bottles [MODIFIED]

This interaction involves the customer scheduling a refill, with the system coordinating the pickup/delivery and inventory updates through the logistics service.

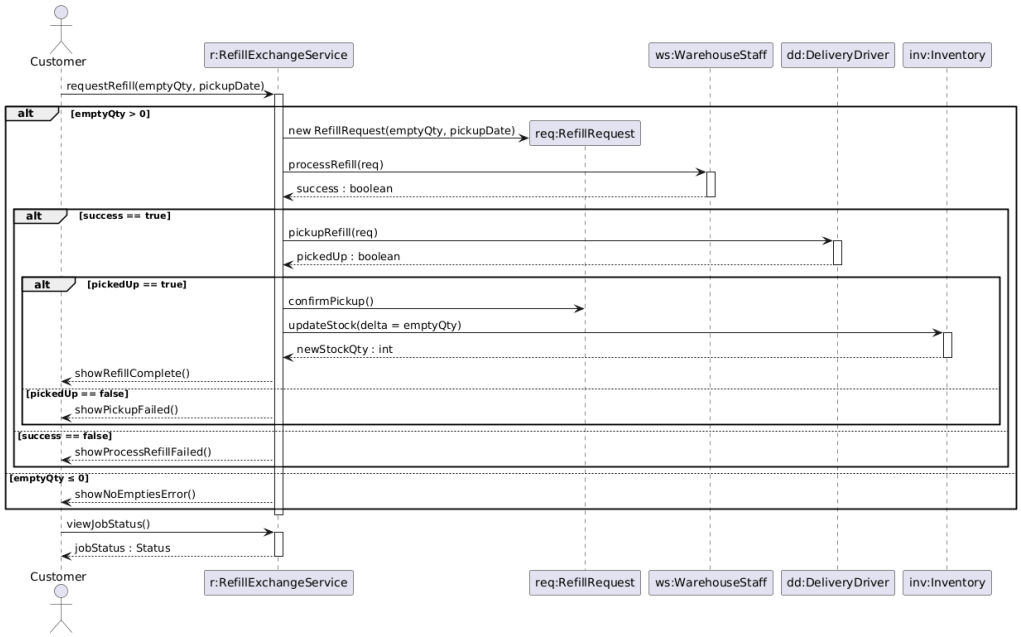


Figure 18: Sequence Diagram — Refill / Exchange (Mohammad Obiad)

Mohmmad ALabed's Diagram: Return / Refund for Damaged Product [MODIFIED]

This scenario details how a customer submits a damage claim with photos. The system routes it to support, who review and issue a refund or replacement.

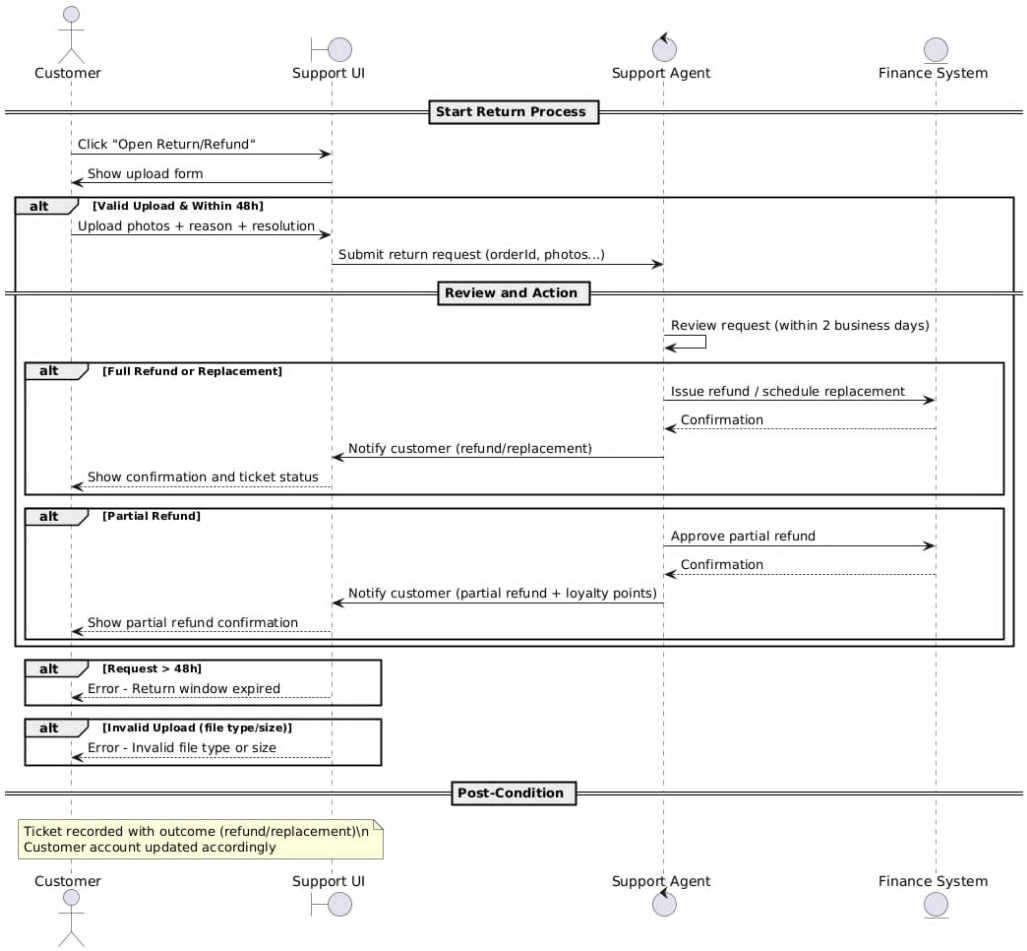


Figure 19: Sequence Diagram — Return / Refund (Mohmmad ALabed)

4 Chapter 4: System Design and Modelling

4.1 Description of Chosen Design Goals [MODIFIED]

Team Lead: [Ahmad Elayyan], Contributing Members: All members (Review, Edit)

This section outlines the key design goals that guided the development of the Online Water Shop system. We selected two general software engineering goals (Low Coupling and High Cohesion), and one specific non-functional system requirement focused on usability. These goals are addressed and visually supported through the class, component, and deployment diagrams.

I. Low Coupling

Objective: Minimize dependencies between modules to improve maintainability, enable easier testing, and allow individual system components (like order management and payment processing) to evolve independently.

Implementation in Our System:

- Classes like `OrderService`, `PaymentProcessor`, and `InventoryManager` communicate through interfaces and service contracts rather than tight bindings.
- Message-passing and service delegation are used to notify components (e.g., delivery confirmation triggers stock update via Logistics API).

Addressed In:

- **Class Diagram:** Shows clear separation between modules with minimal associations.
- **Component Diagram:** Depicts service interfaces and independent deployment units.

II. High Cohesion

Objective: Group related functionalities into tightly focused classes/modules to increase code clarity, robustness, and scalability.

Implementation in Our System:

- The `InventoryManager` is fully responsible for checking stock, reserving units, and updating stock levels—ensuring all inventory logic is centralized.
- The `PaymentGatewayAdapter` encapsulates all logic related to external payment providers (PayPal, Visa, etc.).

Addressed In:

- **Class Diagram:** Highlights high functional grouping per class with clear method boundaries.
- **Sequence Diagram:** Illustrates self-contained workflows where each actor or class sticks to a specific responsibility.

III. Specific Design Goal – User-Friendliness

Objective: Ensure that the system is intuitive and usable by customers and staff with minimal training (target: ≤ 2 days for full system orientation).

Implementation in Our System:

- Customer UI was designed with mobile-first principles and includes simplified steps for browsing, ordering, and tracking.
- Admin dashboards are segmented by roles (manager, delivery staff, support) with minimal interface clutter and guidance messages.
- Error messages, tooltips, and visual feedback (like stock availability and order status) enhance navigation and reduce confusion.

Addressed In:

- **Deployment Diagram:** Highlights how client UIs and staff dashboards are served via distinct front-end components.
- **Component Diagram:** Demonstrates the use of dedicated UI modules and how they connect with backend services using APIs.

System Design Goals [MODIFIED]

Team Lead: [Ahmad Elayyan], Contributing Members: All members (Review, Edit)

In designing the Online Water Shop system, we carefully considered several critical design goals that guide the structure and behavior of the final implementation. These goals ensure that the software is modular, maintainable, scalable, and aligned with both general software engineering principles and specific non-functional requirements.

1. High Cohesion

Objective: Ensure each component or module focuses on a single, well-defined responsibility to enhance maintainability and reduce complexity.

Implementation: In our system, services such as `OrderManager`, `DeliveryScheduler`, and `InventoryService` are structured to handle their respective domains independently. For example, the `DeliveryScheduler` handles only the scheduling of delivery windows and communicates with the `RouteOptimizer`, without being involved in payment or pricing logic.

Design Reflection:

- Use cases like “Schedule Delivery” and “Refill/Exchange Bottles” are implemented through highly cohesive service layers, where each class supports a specific function.
- The discount logic (SR4.1) is decoupled from UI and order submission, enabling reuse in promotions and loyalty modules.
- High cohesion is clearly visible in the **class diagram**, where methods within each class are aligned with a single core responsibility.

2. Low Coupling

Objective: Reduce interdependencies between modules to allow flexibility and easier modification without cascading impacts.

Implementation: The system is architected around loose integration using APIs and service contracts. For example, the `CheckoutUI` calls the `PricingEngine` via a REST interface without needing to understand how discounts are internally processed.

Design Reflection:

- REST APIs serve as boundaries between services such as inventory validation and CRM, minimizing cross-module knowledge.
- Internal logic is hidden using interface definitions and encapsulation, following the principle of information hiding.
- Low coupling is enforced in the **component diagram**, where independent modules are clearly delineated and linked via interface dependencies only.

3. Specific System Design Goal – Performance (Processing Speed and Response Time)

Objective: Ensure a responsive system that completes 95% of REST API calls within 500ms, and address validation under 5 seconds, as defined by non-functional requirements NFR-P1 and NFR-P2.

Implementation:

- Performance-critical services such as bottle lookup, delivery slot selection, and discount calculations are isolated into dedicated microservices.
- In-memory caching is used for frequent reads (e.g., bottle data from master table updated every 5 seconds – SR2.1).
- Asynchronous queuing is implemented for non-blocking operations like sending confirmation SMSes (SR1.1, UR1).

Design Reflection:

- The **deployment diagram** maps performance-sensitive services to separate hardware nodes, reducing contention and ensuring scalability.
- The **component diagram** shows how asynchronous operations and service separation contribute to faster execution under load.

4.2 Component Diagram of System Components [MODIFIED]

Team Lead: Mohammed obied , Contributing Members: All members (Review, Edit)

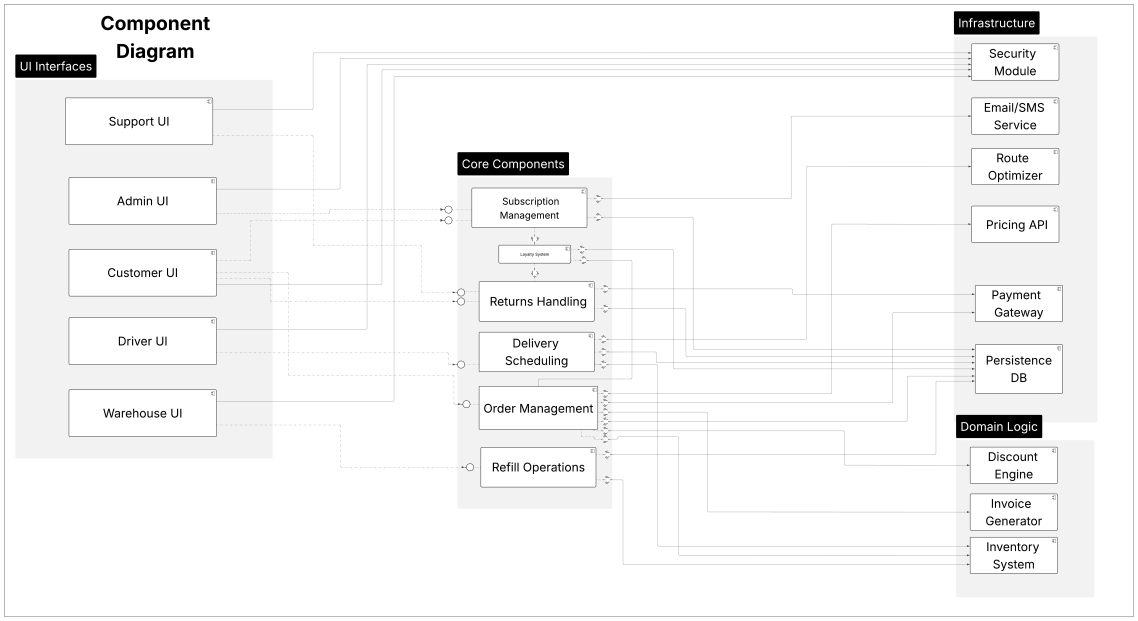


Figure 20: Component Diagram — Online Water Shop System

The component diagram below represents the modular architecture of the **Online Water Shop** system. The architecture is split into four major zones: *UI Interfaces*, *Core Components*, *Infrastructure Services*, and *Domain Logic*. Each layer encapsulates related responsibilities and communicates through well-defined interfaces. This design enforces **low coupling**, **high cohesion**, and enhances overall **system performance and scalability**, as detailed in our design goals (Section 4.1).

- **UI Interfaces:**
  - Separate UIs are provided for each user role: Customer, Driver, Warehouse Staff, Admin, and Support.
  - These UIs interact only with the relevant services (e.g., Driver UI communicates with Delivery Scheduling).
- **Core Components:**
  - Each major use-case is mapped to a dedicated module such as: `Order Management`, `Refill Operations`, `Delivery Scheduling`, and `Returns Handling`.
  - `Subscription Management` contains reusable logic such as `Loyalty System` for reward calculations.
  - These modules are cohesive and well-isolated from one another.
- **Infrastructure Services:**
  - External integrations (e.g., Email/SMS notifications, Route Optimizer, Payment Gateway, Pricing API) are abstracted as infrastructure services.

    * The `Security Module` and `Persistence DB` provide cross-cutting concerns like authentication and storage.
  - **Domain Logic:**
    * Business-critical logic is encapsulated in reusable engines such as the `Discount Engine`, `Invoice Generator`, and `Inventory System`.
    * These services are shared across multiple core components but maintain strict interface boundaries.

This component structure directly supports our design goals:

- **High Cohesion:** Each module is responsible for a single domain concept (e.g., Delivery Scheduling or Refill Handling).
- **Low Coupling:** Communication between components is done via defined interfaces and APIs, which enables easy maintenance and flexibility.
- **Performance:** Infrastructure and domain logic are separated, allowing for asynchronous processing and optimized deployment per service.

4.3 Overall Architecture Diagram [MODIFIED]

Team Lead: Ahmad Elayyan, Contributing Members: All members (Review, Edit)

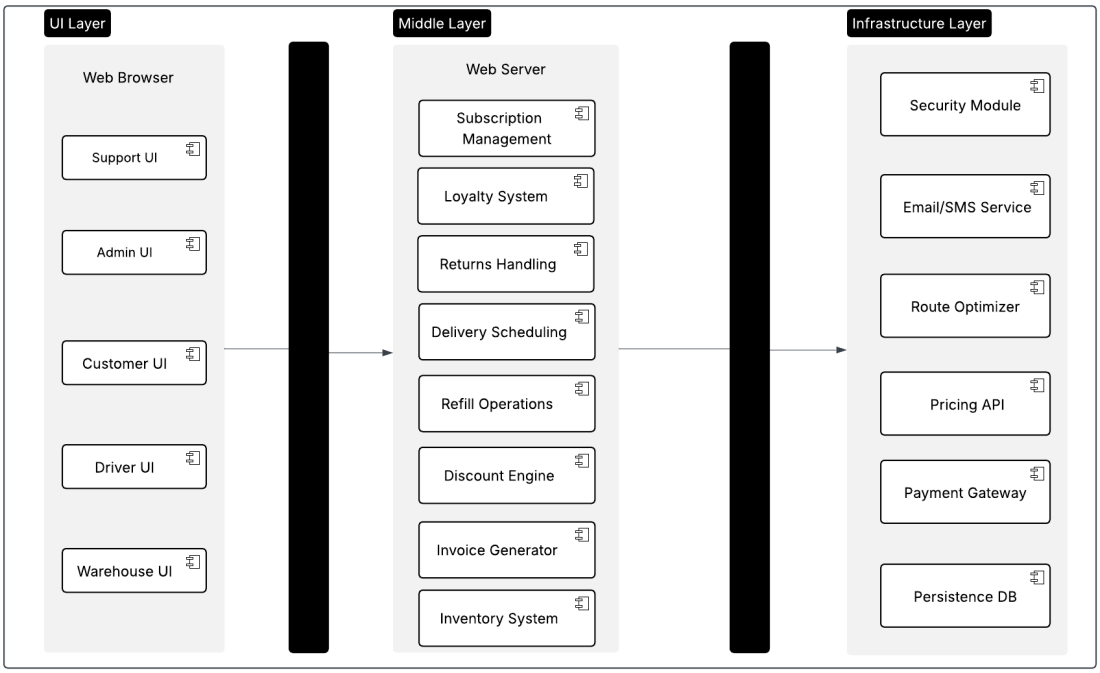


Figure 21: Overall Software Architecture – Online Water Shop (Layered Model)

The figure below presents the high-level software architecture for the **Online Water Shop** system. We have adopted a **Layered Architecture** pattern to separate concerns, improve maintainability, and support scalability. The architecture is logically divided into three main layers: the **UI Layer**, the **Middle Layer (Business Logic)**, and the **Infrastructure Layer**.

Justification for Layered Architecture:

- **Separation of Concerns:** Each layer handles a distinct responsibility — UI, business processing, or system-level infrastructure — allowing for high cohesion within each layer.
- **Supports Low Coupling:** The layers interact via defined APIs, enabling services (like Payment Gateway or Pricing API) to evolve without affecting upper layers.
- **Scalability and Performance:** The middle layer can be scaled horizontally across servers under high load, and infrastructure services can be distributed independently.
- **Maintainability:** Developers can work on one layer (e.g., UI improvements or payment logic) with minimal risk to other layers.
- **Security and Deployment Flexibility:** The separation also makes it easier to apply security measures and deploy specific layers to different environments (e.g., cloud-native microservices).

Layer Breakdown:

- **UI Layer:** Includes all client-facing UIs — Customer, Admin, Support, Driver, and Warehouse — accessed through web browsers.
- **Middle Layer (Web Server):** Contains the core services such as Order Management, Delivery Scheduling, Returns Handling, and shared business logic like the Discount Engine and Loyalty System.
- **Infrastructure Layer:** Hosts external services and system-wide dependencies, including the Payment Gateway, SMS/Email Service, Route Optimizer, and the main database.

4.4 Deployment Diagram [MODIFIED]

Team Lead: Abdallah AboShoaib, Contributing Members: All members (Review, Edit)

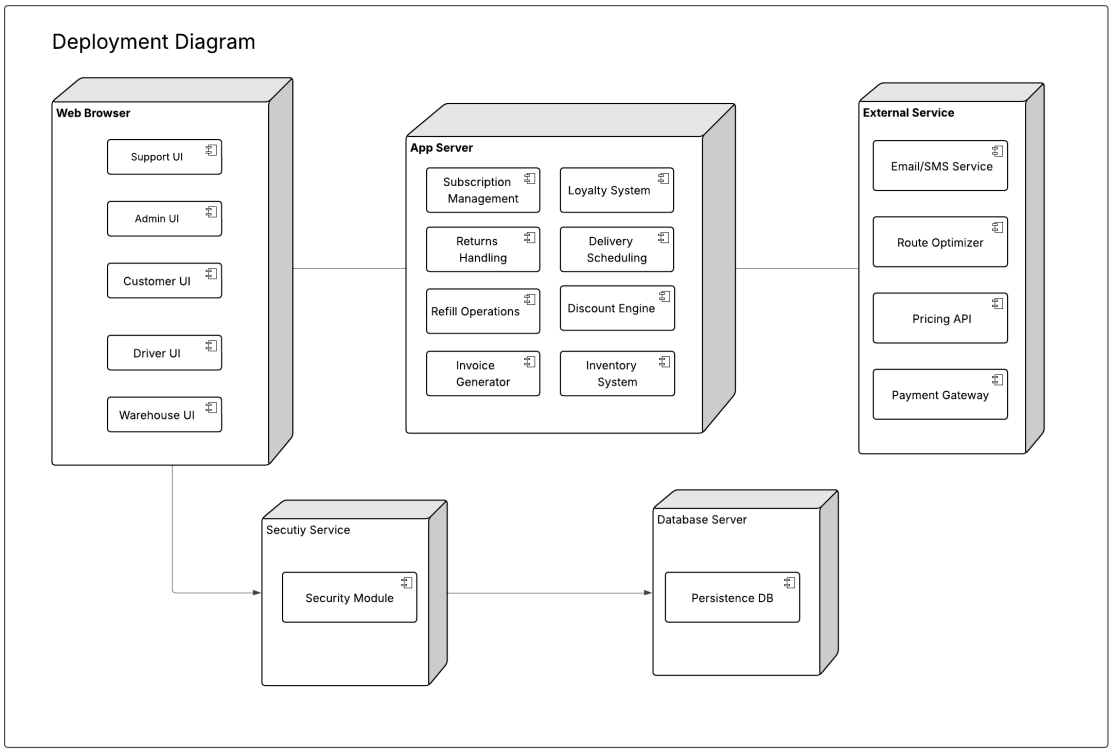


Figure 22: Deployment Diagram – Online Water Shop System

The deployment diagram below illustrates how the major software components of the **Online Water Shop** system are mapped to the physical (or virtual) hardware infrastructure. The system is deployed across five key nodes, each responsible for a group of related services. This design supports our performance, scalability, and security goals discussed in Section 4.1.

Deployment Nodes and Responsibilities:

- **Web Browser:** Hosts the frontend user interfaces for all roles — Customer, Admin, Driver, Warehouse, and Support. These are single-page applications that consume REST APIs from the App Server.
- **App Server:** Acts as the core logic and orchestration node. It runs all business logic including:
  - Order Management, Refill Operations, Returns Handling
  - Loyalty Subscription Management
  - Discount Engine, Invoice Generator, Inventory System
  - Delivery Scheduling

  This centralized processing node ensures all critical services are co-located for fast inter-service communication.

- **External Services:** A logical grouping of all external integrations:
  - Payment Gateway
  - Email/SMS Notification Service
  - Route Optimizer
  - Pricing API

  These are accessed asynchronously and often in parallel to reduce latency (supports our performance goal from Section 4.1).
- **Security Service:** Hosts the centralized authentication and authorization module. This ensures secure access and API protection without embedding security logic within every component.
- **Database Server:** Stores all persistent data including users, orders, product inventory, subscription history, and transaction logs. All services interact with this node through the `Persistence DB`.

Design Rationale:

- **Supports Low Coupling:** Each deployment node encapsulates a distinct concern (e.g., UI, core logic, persistence, security).
- **Performance Focused:** App Server is optimized for high-speed business logic; asynchronous calls to external services prevent blocking.
- **Scalable:** Each node can be horizontally scaled independently based on load.
- **Secure:** Security logic is centralized and protected in a separate service.

Appendix-I: Minutes of Meetings

Team: G1 – Online Water Shop
Prepared by: Ahmad Elayyan (Team Leader)

Below is a detailed log of all meetings held by our team throughout the Software Engineering project. These sessions were essential in aligning our technical direction, clarifying customer requirements, discussing UML diagrams, planning deliverables, and reviewing feedback from Professor Adel.

Date	Meeting Type	Start Time	Duration
07/04/2025	Online	12:05 PM	40 min
13/04/2025	Face-to-Face	1:00 PM	3 hours
19/04/2025	Online	7:48 PM	2 hours 30 min
25/04/2025	Online	7:00 PM	4 hours
01/05/2025	Online	5:30 PM	2 hours
06/05/2025	Online	7:30 PM	3 hours
10/05/2025	Online	5:00 PM	2 hours 30 min
14/05/2025	Online	8:40 PM	3 hours
18/05/2025	Face-to-Face	11:00 AM	2 hours
22/05/2025	Online	5:00 PM	3 hours 30 min
25/05/2025	Online	1:00 PM	2 hours 30 min
25/05/2025	Online	9:00 PM	1 hour
01/06/2025	Face-to-Face	3:00 PM	1 hour
05/06/2025	Online	7:00 PM	3 hours
08/06/2025	Online	7:15 PM	2 hours
11/06/2025	Face-to-Face	11:00 AM	1 hour

Main Discussion Topics:

- Requirement analysis and refining use-cases based on customer group feedback.
- Sequence and activity diagram alignment with selected scenarios.
- Class diagram consistency (attributes, operations, responsibilities).
- Splitting effort estimation and scheduling tasks.
- Iterative reviews of component, architecture, and deployment diagrams.
- Final report formatting and preparation for submission.

These meetings strengthened our collaboration, enhanced our analytical skills, and taught us the value of structured teamwork under deadlines.

Course Feedback

The **Software Engineering (COMP433)** course has been one of the most valuable and impactful learning experiences in our academic journey. Through this course, we didn't just learn theoretical principles — we had the opportunity to apply them in a full-scale, real-world project: the development of the **Online Water Shop** system.

Throughout the semester, we explored every phase of the software engineering lifecycle, including:

- Requirements elicitation and analysis through use-case scenarios.
- High-level and detailed UML modeling (Class, Sequence, Activity, Component, Deployment diagrams).
- System architecture and effort/time estimation using realistic developer workflows.
- Non-functional design considerations such as performance, usability, cohesion, and coupling.

What made this course truly exceptional was the consistent and hands-on guidance of **Prof. Adel Taweel**. His feedback was always detailed, constructive, and grounded in industry best practices. He helped us think like real engineers — not just students completing an assignment. His review of our diagrams, comments on use-case flows, and challenge prompts pushed us to improve our work week after week.

Thanks to this course:

- We now feel confident in participating in actual software development projects.
- We developed key professional skills — communication, version control, task delegation, and iteration under pressure.
- We better understand how good system design leads to maintainable, scalable, and robust software.

We would like to express our deepest appreciation to Prof. Adel for his tireless dedication, clarity in lectures, and encouragement throughout the course. This project has not only enriched our academic record — it genuinely prepared us for real industry environments.

Thank you for helping us build the mindset of future engineers.